

TFM Churn E-commerce — Bitácora de avance (Parte 2)

Continuación de `TFM Churn E-commerce – Bitácora de avance` (Parte 1). Registra las decisiones, código y validaciones realizadas después del cierre de "Paso 0 — Entorno y verificación" y la auditoría de datos.

Nota de alineación de estructura: el plan original de la Parte 1 preveía `src/split/` para el split temporal. En la implementación real se usó `src/splitting/` (y `src/labeling/` para el etiquetado, como ya estaba previsto). Se deja constancia aquí para que la estructura documentada coincida con el repositorio real.

9. Etiquetado de churn — `src/labeling/churn.py`

Función principal: `build_churn_labels(df_sales, cutoff_date, horizon_days)`.

Definición implementada:

- `churn = 1` si el cliente **no** tiene ninguna compra válida en `(cutoff_date, cutoff_date + horizon_days]`.
- `churn = 0` si tiene al menos una.
- Las cancelaciones (ya excluidas por `get_clean_sales`) no cuentan como compra.

Cobertura de tests: `tests/test_churn.py`, incluyendo caso con y sin compra futura. Todos los tests en verde.

Validación con datos reales

Corte `2011-09-10`, horizonte 90 días:

Métrica	Valor
Clientes evaluados	5.281
churn = 1	56,64 %
churn = 0	43,36 %

Comparación de horizontes (60 / 90 / 120 días)

Para comparar los tres horizontes sin censura por la derecha (el dataset termina el `2011-12-09`), se usó una **fecha de corte común**: `2011-08-11` (= fecha máxima – 120 días), de forma que los tres horizontes tuvieran la misma base de clientes y ventana de observación completa.

Horizonte	Clientes	% Churn	% Retención
60 días	5.179	70,73 %	29,27 %
90 días	5.179	60,78 %	39,22 %
120 días	5.179	53,21 %	46,79 %

Decisión metodológica final: H = 90 días como horizonte principal, con 60 y 120 documentados como análisis de sensibilidad.

Justificación:

- Equilibrio estadístico razonable (60,78 % / 39,22 %), menos extremo que H=60.
- Utilidad de negocio: 3 meses de margen real para una campaña de retención.
- Menor riesgo de falsos positivos que H=60; mayor capacidad de intervención que H=120.

10. Features RFM y comportamentales — `src/features/rfm.py`

Función principal: `build_rfm_features(df_sales, cutoff_date)`. Usa únicamente compras con `InvoiceDate <= cutoff_date` (sin fuga de información).

Features generadas:

- `recency`: días desde la última compra hasta el corte.
- `frequency`: número de facturas distintas.
- `monetary`: gasto total (`(Quantity * Price)`).

- `n_products`: productos distintos (`StockCode`).
- `n_countries`: países distintos.
- `avg_days_between_purchases`: intervalo medio entre facturas (NaN si el cliente tiene una sola compra observable — pendiente de imputación explícita en el pipeline de sklearn).

Cobertura de tests: `tests/test_rfm.py`, incluyendo un test anti-leakage (una compra futura no debe alterar ninguna feature ya calculada).

Incidencias resueltas

Problema	Causa	Solución
<code>ValueError: Faltan columnas obligatorias: ['UnitPrice']</code>	El dataset real usa la columna <code>Price</code> , no <code>UnitPrice</code> (nombre heredado del dataset antiguo "Online Retail" 2010)	<code>rfm.py</code> adaptado para usar <code>Price</code> , respetando el contrato de nombres originales de UCI ya establecido en <code>cleaning.py</code>
Discrepancia de 4 clientes entre <code>frequency==1</code> (1.577) y nulos en <code>avg_days_between_purchases</code> (1.573)	Una misma factura con líneas registradas en timestamps distintos (segundos de diferencia, según se escanean los artículos en caja) se contaba como dos compras al agrupar por <code>InvoiceDate</code> completo	Cálculo de intervalos reescrito para agrupar por <code>["Customer ID", "Invoice"]</code> (una fecha representativa por factura), coherente con el cálculo de <code>frequency</code> . Añadido test de regresión <code>test_same_invoice_with_different_timestamps_is_one_purchase</code>

Validación con datos reales

Corte `2011-09-10`:

Chequeo	Resultado
Cientes con features	5.281
<code>monetary</code> mínimo / máximo	1,55 / 484.615,10 (sin valores ≤ 0)
Cientes con <code>frequency == 1</code>	1.577
Nulos en <code>avg_days_between_purchases</code>	1.577 (coincide exactamente tras el fix)
Coherencia con <code>build_churn_labels</code> (mismo corte)	100 % — mismos <code>Customer ID</code> en features y labels

11. Unión features + etiqueta — `src/features/pipeline.py`

Función principal: `build_dataset(df_sales, cutoff_date, horizon_days)`. Combina `build_rfm_features` + `build_churn_labels` en un único dataset cliente + snapshot, validando que el número de filas coincida tras el `merge` (si no coincide, lanza `ValueError` con el detalle).

Cobertura de tests: `tests/test_pipeline.py`.

Validación con datos reales

Corte `2011-09-10`, H=90: 5.281 filas, distribución de `churn` 56,64 % / 43,36 % — coincide exactamente con el resultado de `build_churn_labels` en solitario. Dataset combinado confirmado.

12. Snapshots mensuales — `src/features/snapshots.py`

Para pasar de un único corte a un dataset con múltiples observaciones por cliente (cliente + snapshot), se implementaron:

- `generate_snapshot_dates(min_date, max_date, horizon_days, freq="MS")`: genera cortes mensuales, respetando el mismo criterio anti-censura que la comparación de horizontes (el último snapshot no puede dejar menos de `horizon_days` días de futuro disponibles).
- `filter_active_customers(df_sales, cutoff_date, lookback_days=365)`: en cada snapshot, se conservan solo los clientes con al menos una compra en los 365 días previos al corte (decisión tomada explícitamente para no arrastrar clientes inactivos con features cada vez más planas).
- `build_multi_snapshot_dataset(df_sales, snapshot_dates, horizon_days, lookback_days)`: itera sobre las fechas y concatena los datasets cliente+snapshot, reutilizando `build_dataset`.

Cobertura de tests: `tests/test_snapshots.py`.

Validación con datos reales

Métrica	Valor
Nº de snapshots (dic-2009 a sep-2011, H=90)	22
Filas totales	71.281
Clientes únicos	5.249

El primer snapshot (`2009-12-01`) genera solo 1 fila — apenas hay histórico de 12 meses en ese punto del dataset; se documenta como explicación del crecimiento rápido de volumen en los primeros meses, no como anomalía.

13. Split temporal — `src/splitting/temporal_split.py`

Funciones: `temporal_split(dataset, date_col, train_end, valid_end)` y `summarize_split(...)` para verificación rápida.

Regla: train = snapshots $\leq$ `train_end`; valid = entre `train_end` y `valid_end`; test = posteriores a `valid_end`. Nunca aleatorio.

Limitación documentada: un mismo `Customer ID` puede aparecer en varios splits (train, valid y test), porque tiene varios snapshots. No constituye fuga de información — cada fila respeta su propio corte — pero introduce dependencia entre observaciones del mismo

cliente, mencionada explícitamente como limitación metodológica.

Cobertura de tests: `tests/test_temporal_split.py`, incluyendo un test que documenta expresamente el comportamiento de clientes repetidos entre splits.

Split elegido (Opción A: test más grande, prioriza robustez de evaluación)

Split	Filas	Clientes únicos	Snapshots	Rango de fechas	% Churn
train	32.537	4.266	13	2009-12 a 2010-12	49,34 %
valid	17.076	4.614	4	2011-01 a 2011-04	64,51 %
test	21.668	4.796	5	2011-05 a 2011-09	58,70 %

14. Hallazgo: deriva temporal en la tasa de churn

Al desagregar el `% churn` por snapshot individual se detectó una deriva mucho más amplia que un simple efecto navideño puntual:

Periodo	% Churn
ene-2010	36,86 %
jul-2010 (pico intermedio)	51,33 %
dic-2010 / ene-2011 (pico máximo)	66,92 % / 67,08 %
sep-2011 (final de la serie)	50,50 %

Interpretación de trabajo (no confirmada estadísticamente, pendiente si se quiere profundizar):

- **Subida 2010:** posible efecto de maduración de cohortes — a medida que crece la base de clientes observados (de 955 a 4.266 filas/mes), se incorporan más clientes nuevos/ocasionales, con mayor probabilidad natural de no repetir en 90 días.
- **Bajada 2011:** compatible con la hipótesis estacional — los snapshots de nov-dic-2010 concentran compradores navideños puntuales; esa cohorte se diluye a lo largo de 2011.

Consecuencia para el modelado: con una tasa base tan variable en el tiempo, dos clientes con el mismo RFM pueden tener riesgo real distinto según el momento del ciclo. Se decidió incorporar features estacionales explícitas (ver siguiente sección) en vez de asumir que el problema es estacionario.

15. Features estacionales — `src/features/temporal.py`

Función: `add_seasonal_features(dataset, date_col="cutoff_date")`, integrada al final de `build_dataset` en `pipeline.py`.

Features añadidas:

- `month`: mes calendario del corte (1-12).
- `month_sin` / `month_cos`: codificación cíclica del mes (para que diciembre y enero queden "cerca" en el espacio de features, no en extremos opuestos de una escala lineal).
- `is_post_holiday`: flag = 1 si el corte cae en diciembre-marzo (ventana donde se observó el pico de churn), 0 en caso contrario.

Cobertura de tests: `tests/test_temporal.py`, incluyendo verificación de que enero y diciembre quedan cerca en el espacio cíclico y que el DataFrame original no se muta.

Validación con datos reales

<code>is_post_holiday</code>	% Churn
0 (abr-nov)	53,92 %
1 (dic-mar)	60,36 %

Diferencia de ~6,5 puntos porcentuales en la dirección esperada — confirma que la feature captura señal real, no ruido.

Decisión de diseño pendiente de confirmar en el siguiente bloque: la columna `month` cruda no debe entrar al pipeline de modelado junto con `month_sin`/`month_cos` (sería redundante, y como entero 1-12 induce una escala falsa donde diciembre y enero están "lejos"). Columnas no-predictoras a excluir del `ColumnTransformer`: `Customer ID`, `cutoff_date`, `month`.

16. Estado actual y próximos pasos

✓ Completado en esta sesión

- ✓ `src/labeling/churn.py` — etiquetado de churn con horizonte configurable, testeado.
- ✓ Decisión de horizonte: H = 90 días (con 60/120 como sensibilidad), justificada empíricamente sobre corte común sin censura.
- ✓ `src/features/rfm.py` — features RFM + comportamentales, testeado, con dos bugs reales detectados y corregidos (columna `Price` vs `UnitPrice`; intervalos entre compras a nivel de factura).
- ✓ `src/features/pipeline.py` — unión features + etiqueta en dataset cliente+snapshot.
- ✓ `src/features/snapshots.py` — generación de 22 snapshots mensuales con filtro de clientes activos (lookback 365 días).
- ✓ `src/splitting/temporal_split.py` — split temporal train/valid/test (Opción A), con limitación de clientes repetidos entre splits documentada.
- ✓ Detección y documentación de deriva temporal en la tasa de churn (37 % → 67 % → 50 %).
- ✓ `src/features/temporal.py` — features estacionales (`month_sin/cos`, `is_post_holiday`), señal confirmada con datos reales.

▶▶ Próximos pasos

- ❑ **Decidir tratamiento de clientes mayoristas** (`is_wholesale`, detectado en la auditoría inicial — 70 clientes, 1,19 %): sigue pendiente de la Parte 1, aún no abordado en el bloque de features/labeling. Revisar si conviene excluirlos, modelarlos aparte o incluir `is_wholesale` como feature antes de entrenar.
- ❑ `src/models/` — `ColumnTransformer` + `Pipeline` de preprocesamiento (imputación de `avg_days_between_purchases`, escalado) y `DummyClassifier` como baseline.
- ❑ Regresión Logística → Random Forest → XGBoost, en ese orden, comparando contra el baseline.
- ❑ `src/evaluation/` — PR-AUC, ROC-AUC, precision, recall, F1, calibración y selección de threshold.

- ☐ `src/explainability/` — SHAP sobre el modelo candidato final.
 - ☐ Incorporar MLflow para trazabilidad de experimentos (diferido desde la Parte 1, previsto tras el baseline).
 - ☐ `src/api/` — servicio FastAPI de inferencia.
-

Última actualización: registro generado tras cerrar el ciclo de etiquetado, features RFM, snapshots, split temporal y features estacionales — antes de comenzar el bloque de modelado.